

FreeRTOS Multisensor Room Monitor

Laboratory Report: BCA182 Laboratory 1

Student	John Paul O. Maluya
Course and section	BCA182 / B187.1
Date of submission	Sept 27, 2026
Repository	https://github.com/ilovebeingit/bca182-freertos-multisensor

Abstract

This report presents the design, implementation and verification of a room monitoring node built on an STM32F103C8 “Blue Pill” microcontroller, running native FreeRTOS with the STM32 HAL and simulated in Wokwi. The node measures temperature and humidity with a DHT22, relative ambient light with an LDR module and motion with a PIR sensor. It shows one measurement at a time on a 128x64 SSD1306 OLED, which the user selects with a KY-040 rotary encoder. A 1 kHz buzzer sounds when the temperature leaves the 18.0-30.0 °C band, and the system enters an INACTIVE state after 15 s without motion. Six tasks with justified priorities share the work, and they communicate only through queues, a queue set, an event group and a mutex. All decision logic is written as hardware-independent C++ and covered by 20 host unit tests.

During development the stock FreeRTOS Cortex-M3 port did not run in Wokwi. In-firmware probes showed that the simulator departs from the ARMv7-M architecture in several ways. A project-specific port was therefore designed around the mechanisms that were measured to work. It performs context switches synchronously in the SVC and SysTick handlers, and it implements critical sections by gating the SysTick interrupt and recovering the ticks that fall inside them. With this port, all ten functional requirements were verified in simulation, together with a boot check and three fault experiments; all 14 tests passed. One further anomaly, a stall of the sensor task inside `snprintf`, was observed once. Its cause is inferred but not confirmed.

1. Objectives

The laboratory required a multisensor room monitor to be built on the STM32 Blue Pill with PlatformIO, the STM32Cube framework and native FreeRTOS APIs, without the Arduino framework. The system had to satisfy ten functional requirements, listed in Table 1.

Table 1. Functional requirements.

ID	Requirement
FR-01	Periodic temperature measurement
FR-02	Periodic humidity measurement
FR-03	Ambient light level
FR-04	PIR motion detection
FR-05	The OLED shows one measurement at a time
FR-06	The encoder cycles Temperature → Humidity → Light → Motion clockwise, and the reverse counterclockwise, with wraparound
FR-07	The buzzer sounds when the temperature is outside 18.0-30.0 °C
FR-08	The system has ACTIVE and INACTIVE states
FR-09	No motion for 15 s makes the system INACTIVE
FR-10	Motion makes the system ACTIVE

The ACTIVE state has the OLED on, sensing running and the encoder and alarm enabled. The INACTIVE state has the OLED blank and display work reduced, while the PIR is still monitored.

Beyond these requirements, the design had to observe a number of constraints. Every task must block rather than busy-wait, and at least one periodic task must use `vTaskDelayUntil()`. No intertask data may pass through unsynchronized globals, and shared resources must be protected by a mutex. Every queue, mutex and event group must have a documented role. The hardware-independent decision logic must live in files that include neither HAL nor FreeRTOS headers, so that it can be unit tested on a host PC. At least 13 unit tests were required, with 15 or more as the target. They had to cover the temperature evaluation (five cases), display-mode navigation (four cases including wraparound) and the state transition function (four cases).

The work also served broader learning objectives. These were to structure firmware as cooperating RTOS tasks with priorities justified by timing needs, to choose the appropriate inter-process communication primitive for each job, and to schedule periodic work deterministically. Further objectives were to perform timing-critical bit-banging under an RTOS, to separate hardware logic from decision logic for testing, and to practise engineering hygiene through static analysis, milestone commits and a running development log.

2. System Design

2.1 Hardware

The circuit is defined in `diagram.json` and simulated in Wokwi. Figure 1 shows each peripheral together with the pin and on-chip peripheral it uses, and Figure 2 shows the wired circuit in the simulator.

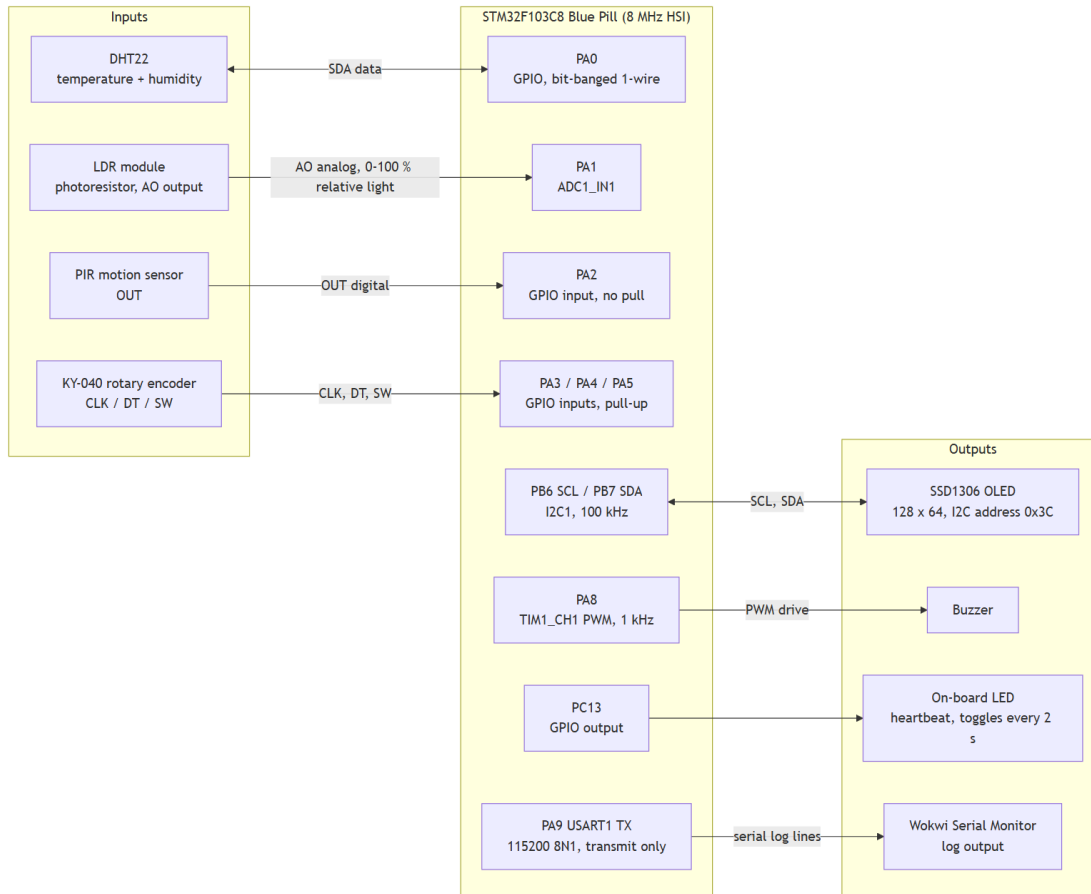


Figure 1. Hardware architecture: the STM32 Blue Pill, its input devices DHT22, LDR module, PIR and KY040 encoder and its output devices SSD1306 OLED, buzzer, heartbeat LED and serial monitor.

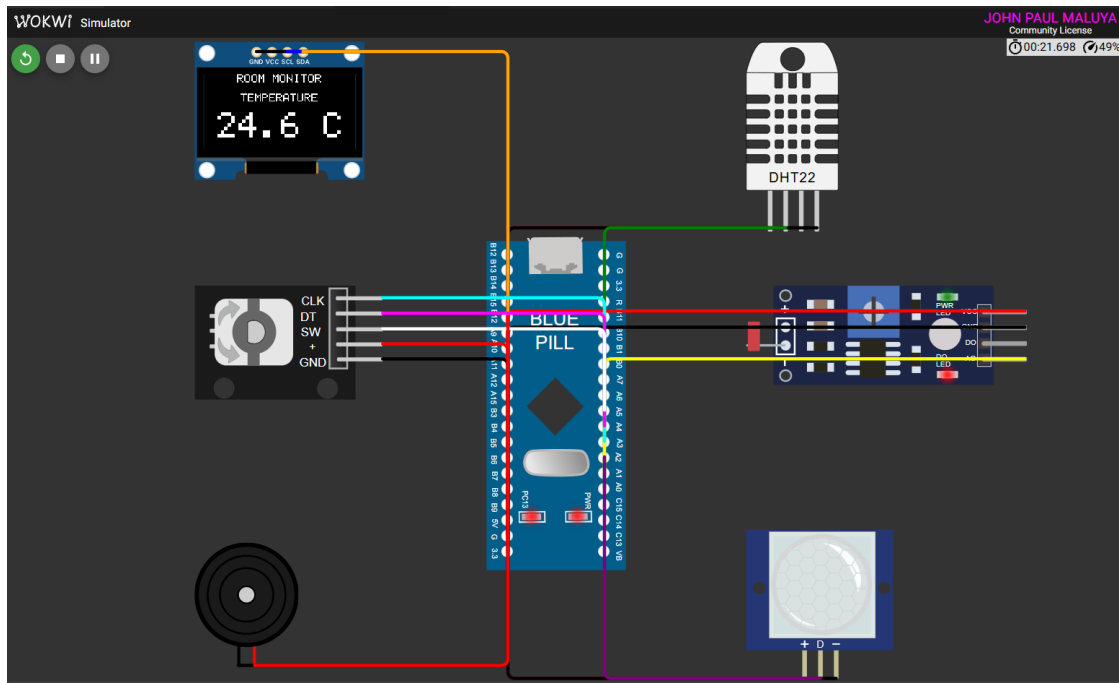


Figure 2. The complete circuit in the Wokwi simulator.

Table 2. Pin assignment.

Pin	Device	Configuration
PA0	DHT22 data	Open-drain output / input with pull-up, switched at register level during a read
PA1	LDR module AO	Analog input, ADC1 channel 1
PA2	PIR OUT	Digital input
PA3 / PA4 / PA5	Encoder CLK / DT / SW	Inputs with pull-up
PA8	Buzzer	TIM1_CH1 alternate function, 1 kHz PWM at 50 % duty
PA9	Serial monitor	USART1 TX, 115200 8N1 (PA10 is wired; the firmware only transmits)
PB6 / PB7	OLED SCL / SDA	I2C1, open-drain alternate function, 100 kHz, address 0x3C
PC13	On-board LED	Push-pull output, heartbeat

A significant finding during design concerned the clock. The buzzer's timer prescaler depends on the timer clock, so the actual clock configuration was examined rather than

assuming the Blue Pill's usual 72 MHz. The examination showed that `SystemClock_Config()` is empty and that `SystemInit()` links as a bare return. No oscillator or clock configuration is linked, and the initial value of `SystemCoreClock` in the image is 8 000 000. The microcontroller therefore runs on its reset clock, the 8 MHz internal HSI oscillator, with the AHB and APB2 buses undivided.

2.2 Software architecture and tasks

The firmware is organized in three layers. The application layer consists of six tasks, one module each. The pure-logic layer consists of `*_logic.cpp` files. These contain the temperature evaluation, state transitions, display-mode navigation, light scaling, DHT22 decoding, and the font engine and screen layout, with no HAL or FreeRTOS dependencies. The platform layer comprises the STM32Cube HAL drivers and FreeRTOS 10.3.1 with the project-specific Cortex-M3 port described in Section 4.

Startup follows a fixed order. `main()` performs the MCU bring-up and calls `app_main()`. This function initializes the hardware, creates all RTOS objects in `rtos_objects_create()`, creates the tasks and starts the scheduler. Any startup failure is reported on the serial log as a `FATAL:` line. The kernel runs preemptively with a 1 kHz tick and uses `heap_4` with a 10 KB heap.

Table 3 lists the six tasks. Priorities were assigned by scheduling urgency. `InputTask` and `MotionTask` run at the highest priority (3), because encoder edges arrive only a few milliseconds apart, and because the PIR drives the return to the `ACTIVE` state and must not wait behind slower work. `SensorTask`, `AlarmTask` and `StateTask` run at priority 2, since they work on 2 s and 15 s timescales. The DHT22's microsecond timing is protected by a critical section rather than by priority. `DisplayTask` runs at the lowest priority (1): a full OLED flush takes about 100 ms of I2C traffic, which a person does not notice, and at this priority it never delays sampling. Every task blocks in each loop iteration and none busy-waits.

Table 3. Tasks.

Task	Trigger	Priority	Stack (words)	Responsibility
<code>InputTask</code>	5 ms, <code>vTaskDelayUntil</code> 1	3	128	Decode the encoder, step the display mode, log button presses
<code>MotionTask</code>	100 ms, <code>vTaskDelayUntil</code> 1	3	128	Sample the PIR and mirror it into <code>EVENT_MOTION</code>
<code>SensorTask</code>	2000 ms, <code>vTaskDelayUntil</code> 1	2	256	Read the DHT22 and LDR, publish a

Task	Trigger	Priority	Stack (words)	Responsibility
AlarmTask	each sample (500 ms timeout)	2	128	SensorData sample, toggle the heartbeat Evaluate the temperature, drive the buzzer, own EVENT_ALARM
StateTask	EVENT_MOTION with a 15 s timeout	2	128	Run the ACTIVE/INACTIV E state machine, own EVENT_ACTIVE
DisplayTask	queue set (250 ms timeout)	1	256	Sole owner of the OLED: render and flush

2.3 Inter-task communication

Figure 3 shows how the tasks communicate. The design contains no global variables for inter-task data. Each hardware handle is static in the module that owns it and is used by a single task. The one exception is the UART, which is shared behind a mutex.

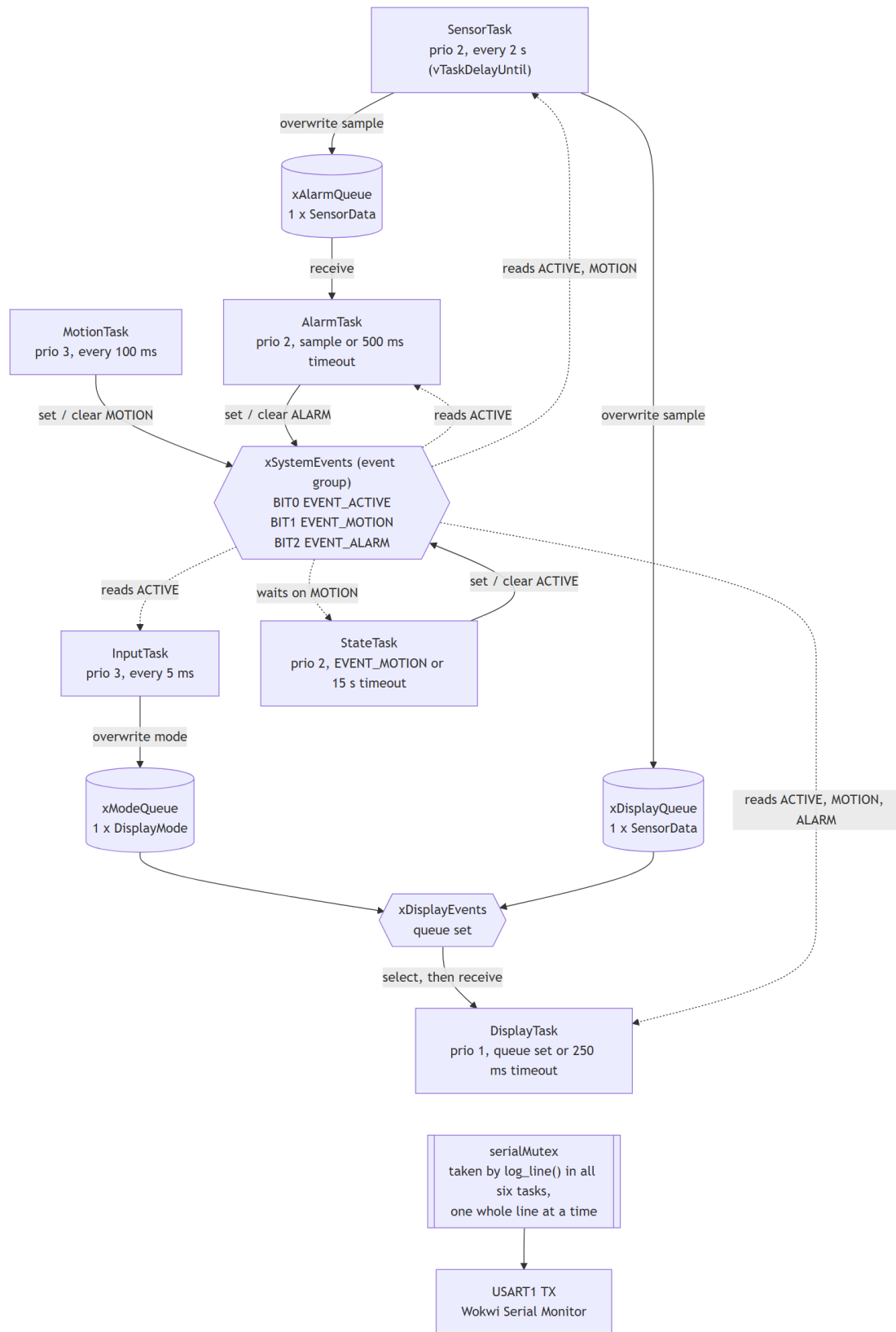


Figure 3. Task communication. Solid arrows denote data sent or event bits written; dotted arrows denote event bits read or waited on.

SensorTask publishes each sample, of type struct SensorData { float temperature; float humidity; int lightLevel; bool motionDetected; }, to two single-item queues, xDisplayQueue and xAlarmQueue. Two queues are needed because a queue item is removed by the task that receives it. With a single queue shared by DisplayTask and AlarmTask, whichever task received first would take the sample and the other would miss it. InputTask passes the selected display mode to DisplayTask through a third single-item queue, xModeQueue. All three queues are written with xQueueOverwrite(): each consumer needs only the newest value, and a producer never blocks.

DisplayTask must wake for whichever arrives first, a new sample or a new mode, but a task can block on only one object. The two queues are therefore joined in the queue set xDisplayEvents, whose length (two) equals the sum of its members' lengths. Event groups cannot be added to a queue set, so DisplayTask waits on the set with a 250 ms timeout and checks the event bits on every wake-up. It redraws only when something shown on screen has changed.

System-wide conditions are held in the event group xSystemEvents. Each of its bits has exactly one writer, as shown in Table 4. An event group was chosen because these conditions are yes/no flags that several tasks need to read at any time, which a queue could not provide, since it hands each item to a single reader. The event group also allows StateTask to block until motion appears, using the remaining inactivity time as the wait's timeout.

Table 4. Event bits.

Bit	Meaning	Set by	Cleared by	Read by
EVENT_ACTIVE (BIT0)	System is ACTIVE	rtos_objects_create() at boot; StateTask on wake	StateTask on timeout	DisplayTask, InputTask, SensorTask, AlarmTask
EVENT_MOTION (BIT1)	PIR reports motion	MotionTask while the PIR is high	MotionTask while the PIR is low	StateTask, DisplayTask, SensorTask
EVENT_ALARM (BIT2)	Buzzer is sounding	AlarmTask when the buzzer starts	AlarmTask when it stops	DisplayTask

Finally, serialMutex protects USART1. HAL_UART_Transmit() is not re-entrant, since its busy check is a non-atomic read followed by a set, so two tasks transmitting at once could drop lines, interleave bytes or corrupt the driver state. The function log_line() holds the mutex for a whole line, including the line terminator. A mutex was preferred to a binary

semaphore because of priority inheritance: a priority-2 task holding the mutex is raised while a priority-3 task waits for it, which bounds the waiting time.

2.4 State machine

Figure 4 shows the ACTIVE/INACTIVE state machine. The system boots ACTIVE. Any motion restarts the 15 s inactivity count, and 15 s without motion moves the system to INACTIVE. Motion seen while INACTIVE returns it to ACTIVE. The transition rule is the pure function `evaluateSystemState(current, motionDetected, timeoutElapsed)`. When motion and an elapsed timeout are seen together, motion takes precedence.

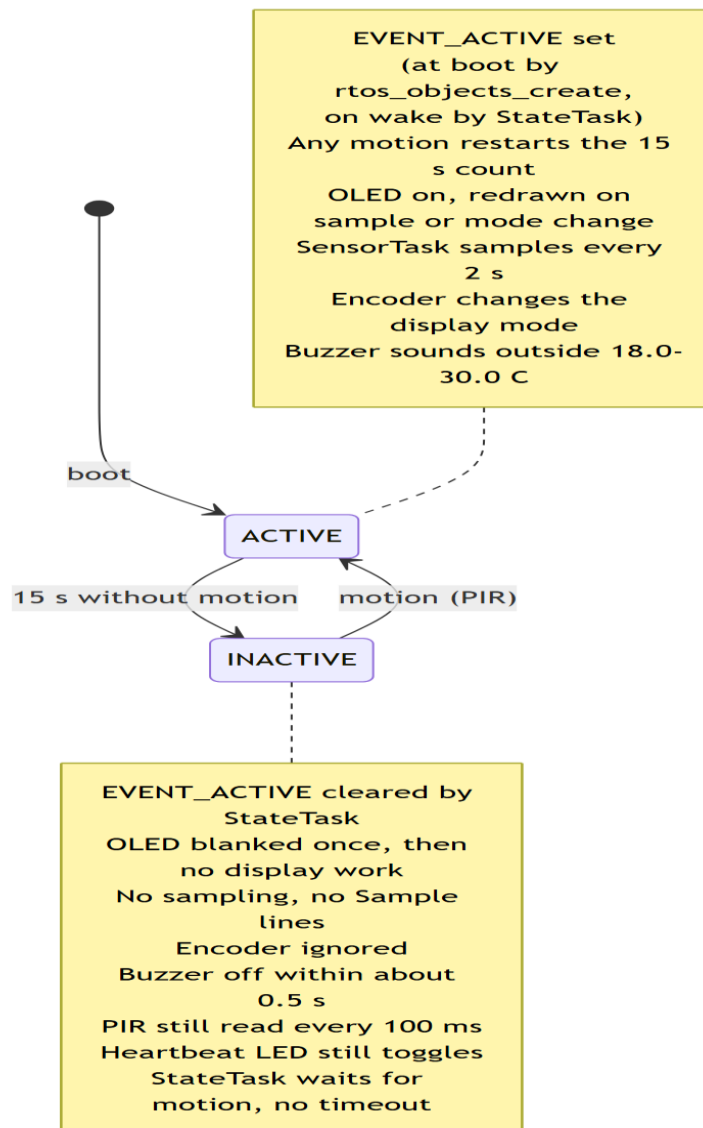


Figure 4. The ACTIVE/INACTIVE state machine and the behaviour of each state.

In the INACTIVE state the OLED is blanked once and receives no further rendering, sensing is skipped (the heartbeat continues), encoder input is ignored and the alarm is silenced, while the PIR remains monitored.

3. Implementation

3.1 Toolchain and structure

The firmware is written in C++ and built with PlatformIO for the bluepill_f103c8 board using the stm32cube framework. FreeRTOS is provided by the STM32Cube Middleware-FreeRTOS library. Its build script compiles only the core kernel files unless optional features are requested, so `custom_freertos_features = event_groups` was added to `platformio.ini`; without it, every event-group call fails to link. Each subsystem has its own module in `include/` and `src/` (sensors, display, input, alarm, motion, system_state and rtos_objects), and `main.cpp` is kept short. The finished build compiles without warnings and uses about 20 KB of flash (31 %) and 12 KB of RAM (59 %), including the 10 KB FreeRTOS heap.

3.2 Sensor acquisition

The DHT22 is read by bit-banging PA0, with pulse widths timed by the Cortex-M3 DWT cycle counter. The 1-2 ms start pulse uses `vTaskDelay`, so other tasks continue to run. Only the 4-5 ms capture phase runs inside `taskENTER_CRITICAL()`, and all it does there is record pulse widths. With the SysTick interrupt gated for that phase, no tick, and therefore no preemptive context switch, can stretch a pulse measurement. Decoding, the checksum test and logging all take place after the critical section ends. Every wait is bounded both by a cycle-count timeout and by an iteration cap, so a missing sensor cannot hang the system.

A failed read skips the whole sampling cycle. Nothing is queued and the reason is logged, so a consumer never receives a sample built from invalid data. The heartbeat LED is toggled on every period regardless.

The LDR module's analog output is sampled on ADC1 channel 1 and converted to a relative 0-100 % light level by `lightPercentFromAdc()`, not to lux. The conversion assumes that the module's output voltage falls as light rises, so a raw reading of 0 maps to 100 % and a raw reading of 4095 to 0 %. This assumption was recorded as unverified during development and was confirmed in functional test FT-03.

3.3 Motion and input

`MotionTask` samples the PIR every 100 ms and mirrors its level into `EVENT_MOTION`, logging each change. `InputTask` samples the encoder every 5 ms. The pure function `encoder_step()` detects a falling edge on CLK and derives the direction from DT. The encoder is decoded continuously, so that no false step appears when the system wakes, but the display mode changes only while the system is ACTIVE. Clockwise steps call

`nextDisplayMode()` and counterclockwise steps call `previousDisplayMode()`, and the result is sent through `xModeQueue`.

3.4 Display

`DisplayTask` is the only task that accesses the OLED and I2C1. A custom 5x7 font engine with integer scaling renders ROOM MONITOR, the measurement label and a large value. The second line shows ALARM LOW or ALARM HIGH while the buzzer sounds. A full flush of the screen to the panel takes about 100 ms of I2C traffic. `DisplayTask` draws nothing before its The firmware is written in C++ and built with PlatformIO for the bluepill_f103c8 board

The firmware is written in C++ and built with PlatformIO for the bluepill_f103c8 board

first sample arrives, and in the INACTIVE state it clears the screen once and then does no further display work.

3.5 Alarm and buzzer

`AlarmTask` evaluates each sample with `evaluateTemperature()`. The function returns LOW below 18.0 °C and HIGH above 30.0 °C; both limits count as normal. The buzzer is driven by hardware PWM on TIM1_CH1. Given the 8 MHz timer clock established in Section 2.1, a 1 kHz tone requires $PSC = 7$ (a 1 MHz timer tick), $ARR = 999$ and $CCR = 500$ for a 50 % duty cycle. `alarm_init()` computes the prescaler from the actual APB2 timer clock at run time, so the tone would stay at 1 kHz if a PLL were configured later, and it logs the values it chose at boot. `AlarmTask` also waits on its queue with a 500 ms timeout. On each wake-up it re-checks `EVENT_ACTIVE`, so the buzzer is silenced soon after the system enters the INACTIVE state.

3.6 State machine task

`StateTask` waits on `EVENT_MOTION` for the time remaining in the 15 s window. Because `EVENT_MOTION` is a level bit that stays set while motion persists, a plain wait on it would return immediately every time, turning `StateTask` into a busy loop, which would violate the blocking requirement. Clearing the bit on exit would stop the spinning, but the other readers would then briefly see “no motion” while motion was present. The adopted solution is a 500 ms hold-off: after seeing motion, `StateTask` blocks with `vTaskDelay` before waiting again. This adds at most 500 ms of latency, well within the 15 s timeout.

3.7 Serial logging

All tasks log through `log_line()`, which holds `serialMutex` for each complete line. Before the scheduler starts only `main()` runs, so `log_line()` skips the mutex at that point, since the mutex may not exist yet. The firmware prints the boot banner BCA182 FreeRTOS Multisensor followed by System starting..., then one line per task start, every sample, every state change and every alarm transition.

3.8 Unit testing and static analysis

The pure logic compiles on the host and is tested with the Unity framework through `pio test -e native`. On Windows, a small pre-build script places PlatformIO's MinGW toolchain on the path, compiles as C++14 like the firmware, and links the runtime statically.

Static analysis with `cppcheck 2.11` (`pio check`) initially reported 72 findings, all of low (style) severity. Two C-style casts in the logging code were fixed. Fifty-eight `unusedFunction` findings were false positives: `pio check` analyses one file at a time and cannot see calls made from other files. That check was therefore disabled for those runs and replaced by a whole-program run, which reported no unused functions and did catch a deliberately planted one. The remaining 12 findings are C-style casts inside CMSIS register macros and FreeRTOS's `xSemaphoreGive()`, not in project code, and were accepted. Every finding is documented in `docs/static-analysis.md`.

4. The Wokwi Scheduler Bug and Its Resolution

4.1 Symptom

When the firmware was first run in Wokwi, only the boot banner appeared. No task ever ran: there was no task-start message, no heartbeat and the OLED stayed blank. A GDB snapshot showed the program counter looping in `prvTaskExitError`, called from `xPortStartScheduler`, in thread mode and without a fault. The kernel reported the scheduler as running with all tasks created, but the process stack pointer held `0xffffffffc` and `CONTROL` was 0, so the first task had never been switched to. `SysTick` interrupts were still being handled, since the tick count kept increasing. `vTaskStartScheduler()` had therefore reached `prvPortStartFirstTask()`, but its `svc 0` instruction never entered the SVC handler, and execution fell through into the “should never get here” path.

4.2 Method

The vector table, `VTOR`, the initial main stack pointer, the `clz` instruction, the interrupt-priority configuration, the heap and the hooks were all ruled out, and a one-task smoke build failed in the same way. GDB reads of special registers proved unreliable in the simulator. The investigation therefore used in-firmware probes: small builds that read their own state through CMSIS intrinsics, recorded the values in RAM and reported them, with GDB used only for plain memory reads and single-stepping.

4.3 Findings

The first probe reproduced `prvPortStartFirstTask()` instruction for instruction. Immediately after the `cpsie i` and `cpsie f` instructions, it read `PRIMASK = 1` and `FAULTMASK = 1`, and its counting SVC handler was entered zero times. When the masks were instead cleared with `msr` before `svc 0`, the handler was entered once. The ARMv7-M

architecture requires CPSIE to clear these masks. In Wokwi it sets them, so the SVC exception was masked and the first task could never start.

Clearing the masks with MSR allowed the first task to start, but nothing ran after it. A probe with hooks at entry to and exit from PendSV recorded 148 157 entries and as many exits, with the handler alternating between two tasks, each always at the same stack depth. Each task's stacked return address was the store instruction that had requested the switch, not the instruction after it. When an exception is triggered by a store that writes ICSR.PENDSVSET, Wokwi stacks the address of the store itself as the return address. The task therefore resumed at the store, re-pended PendSV and switched again, indefinitely.

Further probes, run before FreeRTOS on a minimal two-task switcher, revealed the full set of deviations from ARMv7-M summarized in Table 5.

Table 5. Deviations of Wokwi's Cortex-M3 model from ARMv7-M.

#	Deviation	Effect on the stock port
1	cpsie sets PRIMASK/FAULTMASK and cpsid clears them (inverted)	The first task never starts
2	BASEPRI is not implemented: it reads back 0 and masks nothing	Kernel critical sections protect nothing
3	Exception priorities are stored but not enforced	PendSV (lowest priority) preempts SVC (highest)
4	An exception triggered by a store to ICSR returns to that store	Every yield re-pends PendSV forever
5	A conditional MRS inside an IT block executes even when its condition is false	None; the firmware contains no conditional MRS/MSR
6	PRIMASK and FAULTMASK do not block SysTick	PRIMASK cannot be used for critical sections either

Deviation 4 alone meant that the stock port's context switch, which pends PendSV with a store to ICSR, could not work in Wokwi. Pending PendSV from the SVC handler did not help either, because of deviations 3 and 4. A yield through svc 1 produced more than 325 000 PendSV switches and no task progress. Deviations 2 and 3 meant that every BASEPRI-based critical section, in the kernel, the mutexes, the queues and the DHT22 read, protected nothing. None of these problems is a firmware or configuration fault: real hardware behaves as the architecture specifies.

4.4 What does work

A second series of probes identified the mechanisms that the simulator does emulate correctly. SysTick interrupting a straight-line block of instructions stacked the correct next-instruction address in 64 of 64 samples, so hardware-triggered exceptions are handled correctly and only ICSR-store-triggered ones are wrong. Twenty context switches performed directly inside the SysTick handler, and twenty performed directly inside the SVC handler (svc 1), all alternated correctly between two tasks with register-invariant loops, and no invariant was broken. Clearing the TICKINT bit in SysTick->CTRL stopped the tick handler cleanly, and COUNTFLAG latched a counter wrap that fell inside the gated window. For windows as long as a DHT22 read (4.5 ms, several ticks), COUNTFLAG alone could recover at most one tick. Recovery from the elapsed DWT cycles combined with SysTick's counter value at entry counts the actual tick boundaries, and it was accurate to better than 99 % in the probe run.

4.5 The port

The resolution was a project-specific port, lib/freertos_port_patch/src/port.c with include/portmacro.h, built only on the mechanisms shown to work. It replaces the library's ARM_CM3 port: include/ precedes the library's port directory on the kernel's include path, and the port library is linked as objects rather than as an archive, so the stock port.o is never pulled in.

The port never uses PendSV. portYIELD() calls vPortYield(), which executes svc 1. The SVC handler gates SysTick, saves registers r4-r11 onto the task's stack (the hardware has already stacked the rest), runs vTaskSwitchContext(), restores the next task's registers and returns to it on its own process stack. The SysTick handler performs the same switch when it has interrupted a task and the kernel requests one, which provides time-slicing and wakes delayed tasks. The first task is started with svc 0, after PRIMASK and FAULTMASK have been cleared with MSR. A yield requested inside a critical section, or from an interrupt, is deferred until the critical section ends or the next tick.

Critical sections gate SysTick by clearing TICKINT; the counter keeps running. SysTick is the only interrupt that uses the kernel, so excluding it is sufficient for mutual exclusion. On entry to the outermost level, the port records SysTick's current value and the DWT cycle count. On exit, while the interrupt is still gated, it computes the number of SysTick periods that ended inside the window as $((LOAD - VAL_entry) + cycles_elapsed) / (LOAD + 1)$ and replays that many ticks with xTaskIncrementTick(). Only then does it re-enable the interrupt, so the catch-up can never run concurrently with the tick handler. If a switch is due, it is performed with svc 1. Both the tick handler and every gated window read SysTick->CTRL before proceeding, so COUNTFLAG never counts a wrap twice. The HAL tick, previously advanced from a SysTick wrapper, moved to the FreeRTOS tick hook vApplicationTickHook(), which the kernel calls once per tick, recovered ticks included. The final image was checked to contain no store to ICSR and no access to BASEPRI.

4.6 Verification and limitations

With the new port, the Wokwi terminal showed all six tasks starting, SensorTask sampling every 2 s and StateTask moving the system to INACTIVE after 15 s without motion. A GDB memory snapshot after about 24 s of simulated time agreed. It showed seven tasks (six plus the idle task), with the CPU in the idle task, which means every other task was blocked normally. xTickCount was 24 451 against a HAL tick of 24 462; the 11-tick lead is the HAL counting during boot. The critical-section nesting count was 0, and the last queued sample (24.0 °C, 40.0 %RH, 76 % light) had been consumed by DisplayTask.

One residual limitation is known. A SysTick wrap that falls in the few instructions between gating and reading the counter at entry, or between the last cycle-counter read and ungating at exit, can make the tick count gain or lose one tick. Kernel state is never corrupted. The port is also correct on real hardware, but the library's stock port is simpler and remains the better choice there.

5. Testing

5.1 Unit tests

The unit test suite contains 20 tests in four suites, all passing (Table 6). It exceeds the required minimum of 13 and the target of 15, and it contains each of the required cases. Each suite was checked by deliberately breaking the function under test, for example by changing < to <= at a temperature limit, and confirming that the relevant tests failed.

Table 6. Unit tests.

Suite	Function under test	Cases
test_alarm (5)	evaluateTemperature()	Below the low limit, exactly 18.0 °C, normal, exactly 30.0 °C, above the high limit
test_navigation (6)	nextDisplayMode() / previousDisplayMode()	Forward, backward, wrap MOTION→TEMPERATURE, wrap TEMPERATURE→MOTION, next and previous are inverse, four steps form a full cycle
test_state (5)	evaluateSystemState()	ACTIVE without timeout, ACTIVE with timeout, INACTIVE without motion, INACTIVE with motion, motion overrides an elapsed

Suite	Function under test	Cases
test_light_level (4)	lightPercentFromAdc()	timeout Full scale gives 0 %, zero gives 100 %, mid-scale gives 50 %, out-of-range input is clamped

5.2 Functional tests and fault experiments

Functional testing followed the procedure in docs/test-plan.md. It consists of a boot check (B-01), one functional test per requirement (FT-01 to FT-10) and three fault experiments (F-1 to F-3). Each fault experiment checks that a failure stays contained: the affected function degrades in a defined way while the rest of the system continues to run. All tests were performed in the Wokwi simulation on 2026-09-27, against firmware commit 1e12dc8, and all 14 passed. Table 7 summarizes the observations; the step-by-step records are in the test plan.

Table 7. Functional test and fault experiment results.

Test	Verifies	Observation	Result
B-01	Boot	Banner, timer setup message, all six task-start lines and DISPLAY: OLED initialised, confirmed twice	PASS
FT-01	FR-01	DHT22 set to 11.80 °C; the OLED updated on its own to ALARM LOW / TEMPERATURE / 11.8 C, matching the log	PASS
FT-02	FR-02	Humidity set to 21.00 %; the OLED updated on its own to HUMIDITY / 21.0 %	PASS
FT-03	FR-03	Bright gave 99 % and dark gave 1 %, confirming the conversion direction	PASS
FT-04	FR-04	The OLED showed DETECTED and CLEAR, matching MOTION:	PASS

Test	Verifies	Observation	Result
		detected and MOTION: clear	
FT-05	FR-05	Exactly one label and one value in every mode	PASS
FT-06	FR-06	Full cycle MOTION → LIGHT → HUMIDITY → TEMPERATURE → MOTION with wraparound; the OLED matched each step; two button presses both logged	PASS
FT-07	FR-07	80 °C gave ALARM: temperature HIGH, buzzer on, ALARM HIGH; 11.8 °C gave ALARM: temperature LOW, ALARM LOW	PASS
FT-08	FR-08	Steady 2 s sampling while ACTIVE; sampling stopped immediately at STATE: INACTIVE	PASS
FT-09	FR-09	STATE: INACTIVE (no motion for 15 s) fired correctly several times; exact times not recorded	PASS
FT-10	FR-10	MOTION: detected followed by STATE: ACTIVE (motion) several times; the OLED and sampling resumed each time	PASS
F-1	DHT22 disconnected	DHT22: read failed (no response), sample skipped every 2 s; motion, state and heartbeat unaffected; OLED blank throughout	PASS

Test	Verifies	Observation	Result
F-2	OLED disconnected	DISPLAY: OLED initialised still printed; samples and all other messages continued at the normal pace; OLED dark	PASS
F-3	Alarm while INACTIVE	ALARM: buzzer off immediately after STATE: INACTIVE; on wake the buzzer sounded for about 1 s on the stale 35.2 °C reading until the next sample (24.60 °C) turned it off	PASS

Figure 5 shows a live alarm test. With the DHT22 set to 37.3 °C, the OLED shows ALARM HIGH above the temperature, and the serial terminal shows ALARM: temperature HIGH, the 37.30 °C sample and ALARM: buzzer on.

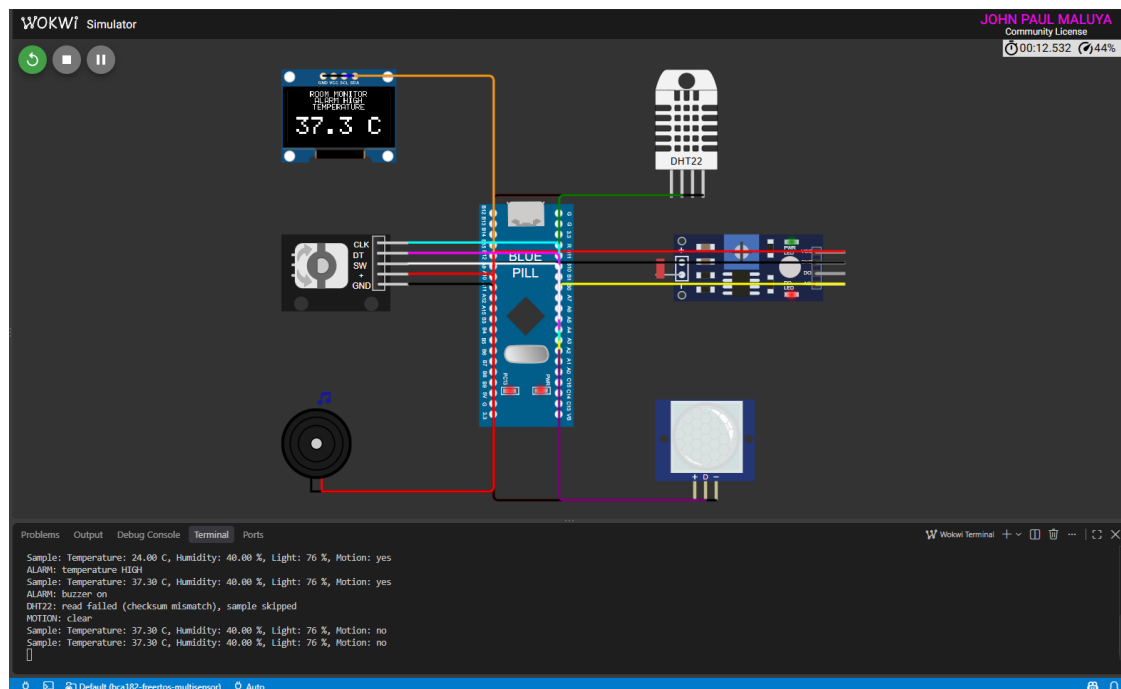


Figure 5. A live alarm test in Wokwi at 37.3 °C: the OLED and the serial terminal side by side.

6. Results and Anomalies

All ten functional requirements were met in simulation, and every fault experiment showed the failure contained as intended. Several observations nevertheless deserve comment.

The exact alarm boundaries of 18.0 °C and 30.0 °C were not stepped through in the simulation, because FT-07 was performed at 80 °C and 11.8 °C. The boundary behaviour is instead established by the five `test_alarm` unit tests, which include both exact limits. Similarly, the FT-06 record lists the counterclockwise sequence, while clockwise navigation and both wraparounds are covered by the navigation unit tests. The buzzer frequency was designed for 1 kHz and the chosen prescaler values are logged at boot, but the tone itself was not measured in the simulation.

Fault experiment F-3 confirmed a predicted side effect of the design. While the system is INACTIVE, no samples are taken, so AlarmTask keeps the last temperature it evaluated. On waking, it therefore sounded the buzzer for about one second on the stale reading, until the next sample corrected it. Experiment F-2 confirmed that an OLED failure is silent. The display driver does not check the results of its I2C transfers, so `DISPLAY: OLED initialised` is printed even with the display disconnected, and nothing in the system reports the failure. The rest of the system was unaffected, which confirms that the lowest-priority, single-owner display design isolates a display fault.

During testing, the OLED was once observed not to refresh on new samples while parked on one display mode. This did not reproduce in FT-01 on the tested build, where the OLED updated on its own, and its cause was not identified.

The most significant anomaly was a stall of SensorTask, observed once while a temporary debug build was running. After one skipped DHT22 read (checksum mismatch), SensorTask produced no further output while the other tasks kept logging. GDB showed the CPU inside `snprintf`, in a padding loop of the newlib printf implementation, with a loop bound of `-2 (0xFFFFFFFF)`. The loop had completed about 10.5 million of roughly 4.29 billion passes, which matches the stall's duration. The format string has no padding, and the values in memory give a pad count of zero. Correct execution cannot therefore produce `-2`. However, if a conditional subtraction inside an IT block (`subeq`) had executed although its condition was false, it would have produced exactly `-2`, the negative of the two digits printed for 24 °C. The hypothesis is that a SysTick exception was taken between the `it` instruction and the `subeq`, and that the simulator lost the IT-block execution state across the exception. On real Cortex-M3 hardware this state is saved in the stacked xPSR and restored on return. The project's port never writes a task's stacked xPSR after creating the task, so the suspected cause is the emulator. This finding was observed once. Its root cause is inferred but was not confirmed by a probe, owing to time constraints. If the hypothesis holds, any task could be affected wherever the compiler emits an IT block, although the one-instruction window makes such events rare. The stalled task also starved the lower-priority DisplayTask and the idle task. Real hardware is not expected to be affected.

Other known limitations are the possible one-tick error at the edges of a critical section (Section 4.6) and the fact that the system has not been tested on real hardware.

7. Discussion

The central lesson of this laboratory is that a simulator is itself a model with its own defects, and that its behaviour must be measured rather than assumed. The initial failure looked like a firmware fault. It was resolved only after the firmware was made to report its own processor state, because the debugger's view of the special registers was itself unreliable. The probes separated the mechanisms that the simulator emulates wrongly from those it emulates correctly. The resulting port uses only the latter: SVC and SysTick return addresses, context switches inside those handlers, and TICKINT gating. This approach turned an apparently blocked project into a working one without depending on any undocumented behaviour. The price was a non-standard port, which must be clearly identified as simulator-specific, since the stock port remains the better choice on real hardware.

The choice of inter-task communication primitives followed directly from the data involved. Latest-value, single-item queues with overwrite semantics suit periodic measurements, since a consumer needs only the newest sample and a producer must never block. The requirement of one queue per consumer reflects the destructive nature of a queue receive. The queue set solved the problem of a single task waiting on two sources. Its inability to include an event group was handled with a short timeout rather than an extra task. Event bits suited shared status flags, and the level semantics of EVENT_MOTION required a hold-off in StateTask to avoid a busy loop. This shows that a blocking call does not by itself guarantee that a task blocks.

The separation of decision logic from hardware proved valuable. It made 20 host tests possible without mocking the HAL or the kernel. Those tests also covered boundary conditions that were not exercised in the simulation, and they gave confidence that the encoder navigation and state machine were correct before the scheduler ran at all. Timing-critical code benefited from the same discipline. Keeping the DHT22 critical section down to pulse capture alone, and bounding every wait, meant that even a disconnected sensor could not hang the system, as experiment F-1 confirmed.

The fault experiments were also informative about what the design does not do. The display failure is contained but silent, and the alarm briefly acts on stale data after waking. Neither violates a requirement, but both are natural candidates for improvement: the unchecked I2C transfer results, and the alarm state carried across the INACTIVE period.

8. Conclusion

A FreeRTOS-based multisensor room monitor was designed, implemented and verified on a simulated STM32 Blue Pill. The system meets all ten functional requirements. It uses six tasks with priorities justified by their timing needs, communicates exclusively through queues, a queue set, an event group and a mutex, and uses `vTaskDelayUntil()` for its periodic tasks. All 14 functional tests and fault experiments passed in simulation, and the 20 unit tests exceed the required count and cover the required cases. The laboratory's most substantial technical contribution was the diagnosis of six departures of the Wokwi Cortex-M3 model from the ARMv7-M architecture, and the design of a synchronous-switch FreeRTOS port that runs correctly despite them. One further simulator anomaly, the loss of IT-block state across an exception, is supported by strong circumstantial evidence but remains unconfirmed. Confirming it with a dedicated probe, testing on real hardware with the stock port, and addressing the silent display failure and the stale alarm state on wake are the natural next steps.

References

1. FreeRTOS Kernel V10.3.1, as packaged in the STM32Cube Middleware-FreeRTOS library for PlatformIO (mincrmatt12/STM32Cube Middleware-FreeRTOS).
2. STMicroelectronics, STM32Cube HAL for the STM32F1 series (PlatformIO stm32cube framework).
3. Arm, *ARMv7-M Architecture Reference Manual*, the architecture against which the Wokwi deviations in Section 4 are described.
4. PlatformIO, <https://platformio.org/>
5. Wokwi, *Wokwi for VS Code*, <https://docs.wokwi.com/vscode/getting-started>
6. Wokwi, *PIR Motion Sensor* part reference, <https://docs.wokwi.com/parts/wokwi-pir-motion-sensor>
7. Unity unit test framework, used through PlatformIO (test_framework = unity).
8. Cppcheck 2.11 static analyser, used through pio check.
9. Project documentation in this repository: docs/requirements.md, docs/design-notes.md, docs/dev-log.md, docs/test-plan.md and docs/static-analysis.md.

Appendix A. Key Code Excerpts

The excerpts below are reproduced from the repository at the time of writing.

A.1 Creation of the RTOS objects

```
(src/rtos_objects.cpp)

bool rtos_objects_create(void) {
    xDisplayQueue = xQueueCreate(1, sizeof(SensorData));
```

```

xAlarmQueue = xQueueCreate(1, sizeof(SensorData));
xModeQueue = xQueueCreate(1, sizeof(DisplayMode));
serialMutex = xSemaphoreCreateMutex();
xSystemEvents = xEventGroupCreate();

/* A set must hold as many events as its members can hold items: 1 + 1.
 * Members must be empty when added, which they are here. */
xDisplayEvents = xQueueCreateSet(2);

if (xDisplayQueue == NULL || xAlarmQueue == NULL || xModeQueue == NULL ||
    serialMutex == NULL || xDisplayEvents == NULL || xSystemEvents == NUL
L) {
    return false;
}

/* The system boots ACTIVE (FR-08). */
xEventGroupSetBits(xSystemEvents, EVENT_ACTIVE);

return xQueueAddToSet(xDisplayQueue, xDisplayEvents) == pdPASS &&
       xQueueAddToSet(xModeQueue, xDisplayEvents) == pdPASS;
}

```

A.2 Periodic sampling with vTaskDelayUntil()

(src/sensors.cpp, excerpt)

```

TickType_t last_wake = xTaskGetTickCount();

for (;;) {
    vTaskDelayUntil(&last_wake, pdMS_TO_TICKS(kSensorPeriodMs));

    HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13); /* heartbeat, every period
*/

    /* FR-08: sensing happens only while ACTIVE. The period keeps running
     * so sampling resumes on schedule once the system is ACTIVE again. */
    /

    if ((xEventGroupGetBits(xSystemEvents) & EVENT_ACTIVE) == 0) {
        continue;
    }

    /* A failed DHT22 read skips this cycle: nothing is sent, so the
     * consumers keep their previous sample and never see a bad value. */
    Dht22Reading reading;
    Dht22Status status = dht22_read(&reading);
    if (status != DHT22_OK) {
        snprintf(line, sizeof(line), "DHT22: read failed (%s), sample ski
pped",
                dht22_status_name(status));

```

```

        log_line(line);
        continue;
    }
    /* ... convert the reading, read the LDR, set motionDetected ... */
    xQueueOverwrite(xDisplayQueue, &sample);
    xQueueOverwrite(xAlarmQueue, &sample);
    /* ... format and log the "Sample:" line ... */
}

```

A.3 Pure decision logic

(src/alarm_logic.cpp, src/system_state_logic.cpp, src/input_logic.cpp)

```

AlarmState evaluateTemperature(float temperatureC) {
    if (temperatureC < kTempLowLimitC) {
        return AlarmState::LOW_TEMPERATURE;
    }
    if (temperatureC > kTempHighLimitC) {
        return AlarmState::HIGH_TEMPERATURE;
    }
    return AlarmState::NORMAL;
}

```

```

SystemState evaluateSystemState(SystemState current, bool motionDetected, bool timeoutElapsed) {
    if (motionDetected) {
        return SystemState::ACTIVE;
    }
    if (current == SystemState::ACTIVE && timeoutElapsed) {
        return SystemState::INACTIVE;
    }
    return current;
}

```

```

int8_t encoder_step(bool prev_clk_high, bool clk_high, bool dt_high) {
    if (prev_clk_high && !clk_high) {
        return (dt_high != clk_high) ? 1 : -1;
    }
    return 0;
}

```

A.4 Alarm task

(src/alarm.cpp, loop)

```

for (;;) {
    SensorData sample;
    if (xQueueReceive(xAlarmQueue, &sample, pdMS_TO_TICKS(kActiveRecheckMs)) == pdTRUE) {
        AlarmState new_state = evaluateTemperature(sample.temperature);
        if (new_state != state) {

```

```

        state = new_state;
        snprintf(line, sizeof(line), "ALARM: temperature %s", alarmStateName(state));
        log_line(line);
    }
}

/* FR-07 alarm, only while ACTIVE (FR-08). */
bool active = (xEventGroupGetBits(xSystemEvents) & EVENT_ACTIVE) != 0;
bool should_sound = active && state != AlarmState::NORMAL;

if (should_sound != sounding) {
    sounding = should_sound;
    buzzer_set(sounding);
    if (sounding) {
        xEventGroupSetBits(xSystemEvents, EVENT_ALARM);
    } else {
        xEventGroupClearBits(xSystemEvents, EVENT_ALARM);
    }
    log_line(sounding ? "ALARM: buzzer on" : "ALARM: buzzer off");
}
}

```

A.5 State task with the motion hold-off

(src/system_state.cpp, loop)

```

for (;;) {
    /* ACTIVE: wait for motion, but only until the 15 s since the last
     * motion run out. INACTIVE: only motion can change anything. */
    TickType_t since = xTaskGetTickCount() - last_motion;
    TickType_t wait = portMAX_DELAY;
    if (state == SystemState::ACTIVE) {
        wait = since < timeout ? timeout - since : 0;
    }

    EventBits_t bits = xEventGroupWaitBits(xSystemEvents, EVENT_MOTION,
                                           pdFALSE, /* MotionTask owns
the bit */
                                           pdFALSE, wait);
    bool motion = (bits & EVENT_MOTION) != 0;

    TickType_t now = xTaskGetTickCount();
    if (motion) {
        last_motion = now;
    }
    bool timeout_elapsed = (now - last_motion) >= timeout;

    SystemState next = evaluateSystemState(state, motion, timeout_elapsed);
}

```

```

    if (next != state) {
        state = next;
        if (state == SystemState::ACTIVE) {
            xEventGroupSetBits(xSystemEvents, EVENT_ACTIVE);
            log_line("STATE: ACTIVE (motion)");
        } else {
            xEventGroupClearBits(xSystemEvents, EVENT_ACTIVE);
            log_line("STATE: INACTIVE (no motion for 15 s)");
        }
    }

    if (motion) {
        vTaskDelay(pdMS_TO_TICKS(kMotionHoldOffMs));
    }
}

```

A.6 Mutex-protected logging

(src/serial_log.cpp)

```

void log_line(const char *msg) {
    bool use_mutex = xTaskGetSchedulerState() != taskSCHEDULER_NOT_STARTED;

    if (use_mutex) {
        xSemaphoreTake(serialMutex, portMAX_DELAY);
    }

    /* HAL_UART_Transmit takes bytes; the text is sent as its raw chars. */
    static const char kNewline[] = "\r\n";
    HAL_UART_Transmit(&huart1, reinterpret_cast<const uint8_t *>(msg),
        static_cast<uint16_t>(strlen(msg)), kUartTimeoutMs);
    HAL_UART_Transmit(&huart1, reinterpret_cast<const uint8_t *>(kNewline),
        sizeof(kNewline) - 1, kUartTimeoutMs);

    if (use_mutex) {
        xSemaphoreGive(serialMutex);
    }
}

```

A.7 Critical-section exit with tick recovery

(lib/freertos_port_patch/src/port.c)

```

void vPortExitCritical( void )
{
    configASSERT( uxCriticalNesting );
    uxCriticalNesting--;

    if( uxCriticalNesting == 0 )
    {

```

```

uint32_t ulNowCycles, ulPeriod, ulMissed;
BaseType_t xSwitchRequired = pdFALSE;

    /* Still gated. Clear COUNTFLAG: the wraps in this window are counted
    exactly below, so the flag must not be counted again later. */
    ulNowCycles = portDWT_CYCCNT_REG;
    ( void ) portNVIC_SYSTICK_CTRL_REG;

    /* SysTick wraps inside the window: cycles since the last wrap before
    entry (LOAD - VAL at entry), plus the cycles elapsed, in whole period
s. */
    ulPeriod = portNVIC_SYSTICK_LOAD_REG + 1UL;
    ulMissed = ( ( ulPeriod - 1UL - ulCriticalEntryVal ) + ( ulNowCycles
- ulCriticalEntryCycles ) ) / ulPeriod;

    /* Catch up while SysTick is still gated, so the tick handler cannot
    run at the same time. */
    while( ulMissed > 0UL )
    {
        if( xTaskIncrementTick() != pdFALSE )
        {
            xSwitchRequired = pdTRUE;
        }
        ulMissed--;
    }

    vPortUngateTick();

    if( ( xSwitchRequired != pdFALSE ) || ( ulYieldPending != 0UL ) )
    {
        ulYieldPending = 0UL;
        __asm volatile( "svc 1" ::: "memory" );
    }
}
}

```

A.8 Synchronous context switch in the SysTick handler

(lib/freertos_port_patch/src/port.c)

```

void SysTick_Handler( void )
{
    __asm volatile
    (
        "    ldr r0, =0xe000e010          \n"
        "    ldr r0, [r0]                \n" /* Read CTRL: clears COUNTFLAG. *
/
        "    mvn r1, #2                  \n"
        "    cmp lr, r1                  \n" /* Interrupted a task? */
        "    bne 1f                      \n"
    )
}

```

```

"    mrs r0, psp                                \n"
"    isb                                         \n"
"    ldr r3, =pxCurrentTCB                      \n"
"    ldr r2, [r3]                              \n"
"    stmdb r0!, {r4-r11}                       \n"
"    str r0, [r2]                              \n"
"    stmdb sp!, {r3, r14}                      \n"
"    bl vPortTickFromTask                      \n"
"    ldmia sp!, {r3, r14}                      \n"
"    ldr r1, [r3]                              \n"
"    ldr r0, [r1]                              \n"
"    ldmia r0!, {r4-r11}                      \n"
"    msr psp, r0                               \n"
"    isb                                         \n"
"    bx r14                                     \n"
"1: b vPortTickNotFromTask                    \n"
"    .ltorg                                     \n"
");
}

```